Java on the Command Line

Learning by Example

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-13

Outline

1. How to Read These Examples
2. Compiling and Running
3. Packages and Many Files
4. Packaging in JARs
5. Errors and Other Tools
6. Summary

How to Read These Examples

Reading a terminal session

- Every grey-black box is a **real session**, run with Java 21 for these slides
- Yellow lines starting with $ are what was **typed**; the rest is what the computer printed
- The sessions ran in a `bash` shell (Linux); **macOS Terminal** (zsh) gives the same results
- **Windows** differences are shown next to the commands where they matter

Each example's files are in `codeoutput/<Name>/`; `bash run_examples.sh` repeats all sessions (macOS: Terminal; Windows: Git Bash).

Terminal `codeoutput/Versions.txt`

```
$ java -version
openjdk version "21.0.12.1" 2026-08-18
OpenJDK Runtime Environment (build 21.0.12.1+1-1-24.04.4-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12.1+1-1-24.04.4-Ubuntu,
        mixed mode, sharing)
$ java --version
openjdk 21.0.12.1 2026-08-18
OpenJDK Runtime Environment (build 21.0.12.1+1-1-24.04.4-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12.1+1-1-24.04.4-Ubuntu,
        mixed mode, sharing)
$ javac -version
javac 21.0.12.1
$ jar --version
jar 21.0.12.1
```

`-version` prints to the error stream (old style); `-version` prints to standard output.

The same commands on Windows

Open **Command Prompt** (`Win`, type `cmd`) or **PowerShell**. The Java commands are identical:

Windows (Command Prompt)

```
C:\Users\ayse\cse101> javac Hello.java
C:\Users\ayse\cse101> dir /b
Hello.class
Hello.java
C:\Users\ayse\cse101> java Hello
Hello, World!
```

macOS (Terminal)

```
ayse@mac cse101 % javac Hello.java
ayse@mac cse101 % ls
Hello.class  Hello.java
ayse@mac cse101 % java Hello
Hello, World!
```

- The **prompt** differs (`C:\...>` versus `... %`); in these slides it is shortened to `$`
- Differences to watch: class-path separator (`;` vs `:`), `type` vs `cat`, `dir` vs `ls`, `%ERRORLEVEL%` vs `$?`

Compiling and Running

Compile, then run

codeoutput/Hello/Hello.java **Java**

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

`javac` takes a **file** (`Hello.java`) and writes `Hello.class`; `java` takes a **class name** (`Hello`, without `.class`).

Compile, then run

codeoutput/Hello/Hello.java — Java

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Terminal — codeoutput/Hello.txt

```
$ ls
Hello.java
$ javac Hello.java
$ ls
Hello.class
Hello.java
$ java Hello
Hello, World!
```

`javac` takes a **file** (`Hello.java`) and writes `Hello.class`; `java` takes a **class name** (`Hello`, without `.class`).

Source-file mode: `java Hello.java`

codeoutput/SourceMode/Hello.java **Java**

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Compiled in memory and run; no `.class` file is left behind (second `ls`). Needs Java 11 or later.

Source-file mode: `java Hello.java`

codeoutput/SourceMode/Hello.java — Java

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Terminal — codeoutput/SourceMode.txt

```
$ ls
Hello.java
$ java Hello.java
Hello, World!
$ ls
Hello.java
```

Compiled in memory and run; no `.class` file is left behind (second `ls`). Needs Java 11 or later.

Command-line arguments: `String[] args`

codeoutput/Args/Args.java — Java

```java
public class Args {
    public static void main(String[] args) {
        System.out.println(args.length + " argument(s)");
        for (int i = 0; i < args.length; i++) {
            System.out.println("args[" + i + "] = " + args[i]);
        }
    }
}
```

The **shell** splits the line at spaces; quotes keep `"two words"` together. **Windows** Command Prompt only understands double quotes: `java Args "Ayşe Yılmaz" -verbose`.

Command-line arguments: `String[] args`

codeoutput/Args/Args.java — Java

```java
public class Args {
    public static void main(String[] args) {
        System.out.println(args.length + " argument(s)");
        for (int i = 0; i < args.length; i++) {
            System.out.println("args[" + i + "] = " + args[i]);
        }
    }
}
```

Terminal — codeoutput/Args.txt

```
$ javac Args.java
$ java Args one "two words" 3
3 argument(s)
args[0] = one
args[1] = two words
args[2] = 3
$ java Args
0 argument(s)
$ java Args 'Ayşe Yılmaz' --verbose
2 argument(s)
args[0] = Ayşe Yılmaz
args[1] = --verbose
```

The **shell** splits the line at spaces; quotes keep `"two words"` together. **Windows** Command Prompt only understands double quotes: `java Args "Ayşe Yılmaz" -verbose`.

Standard input, output and error

codeoutput/Sum/Sum.java — Java

```java
import java.util.Scanner;

public class Sum {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        long sum = 0;
        while (in.hasNext()) {
            if (in.hasNextLong()) {
                sum += in.nextLong();
            } else {
                System.err.println("skipping: " + in.next());
            }
        }
        System.out.println("sum = " + sum);
    }
}
```

| pipe, < read from a file, > write to a file, 2> redirect the **error** stream. These work in Windows Command Prompt too (use `type` instead of `cat`).

Standard input, output and error

codeoutput/Sum/Sum.java — Java

```java
import java.util.Scanner;

public class Sum {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        long sum = 0;
        while (in.hasNext()) {
            if (in.hasNextLong()) {
                sum += in.nextLong();
            } else {
                System.err.println("skipping: " + in.next());
            }
        }
        System.out.println("sum = " + sum);
    }
}
```

Terminal — codeoutput/Sum.txt

```
$ javac Sum.java
$ echo "3 4 5" | java Sum
sum = 12
$ cat numbers.txt
10 20
30
40
$ java Sum < numbers.txt
sum = 100
$ java Sum < numbers.txt > result.txt
$ cat result.txt
sum = 100
$ java Sum < bad.txt
skipping: two
sum = 4
$ java Sum < bad.txt 2> errors.txt
sum = 4
$ cat errors.txt
skipping: two
```

| pipe, < read from a file, > write to a file, 2> redirect the **error** stream. These work in Windows Command Prompt too (use `type` instead of `cat`).

Exit codes: `System.exit(n)`

codeoutput/Exit/Check.java — Java

```java
public class Check {
    public static void main(String[] args) {
        if (args.length != 1) {
            System.err.println("usage: java Check <number>");
            System.exit(2);                    // 2: wrong usage
        }
        int n = Integer.parseInt(args[0]);
        if (n < 0) {
            System.err.println(n + " is negative");
            System.exit(1);                    // 1: failure
        }
        System.out.println(n + " is fine"); // 0: success
    }
}
```

0 means success. Scripts test it: `&&` runs on success, `||` on failure. **Windows**: `echo %ERRORLEVEL%` instead of `echo $?`; an uncaught exception gives exit code 1.

Exit codes: `System.exit(n)`

codeoutput/Exit/Check.java — Java

```java
public class Check {
    public static void main(String[] args) {
        if (args.length != 1) {
            System.err.println("usage: java Check <number>");
            System.exit(2);                    // 2: wrong usage
        }
        int n = Integer.parseInt(args[0]);
        if (n < 0) {
            System.err.println(n + " is negative");
            System.exit(1);                    // 1: failure
        }
        System.out.println(n + " is fine"); // 0: success
    }
}
```

Terminal — codeoutput/Exit.txt

```
$ javac Check.java
$ java Check 5
5 is fine
$ echo $?
0
$ java Check -5
-5 is negative
$ echo "exit code: $?"
exit code: 1
$ java Check
usage: java Check <number>
$ echo "exit code: $?"
exit code: 2
$ java Check 7 && echo "ok" || echo "failed"
7 is fine
ok
$ java Check -7 && echo "ok" || echo "failed"
-7 is negative
failed
```

0 means success. Scripts test it: `&&` runs on success, `||` on failure. **Windows**: `echo %ERRORLEVEL%` instead of `echo $?`; an uncaught exception gives exit code 1.

System properties (-D) and environment variables

codeoutput/Props/Props.java — Java

```java
public class Props {
    public static void main(String[] args) {
        String g = System.getProperty("greeting", "Hello"); // -D
        String c = System.getenv("COURSE");                 // env
        System.out.println(g + " from " + (c == null ? "?" : c));
        System.out.println(System.getProperty("os.name")
                + ", path.separator = "
                + System.getProperty("path.separator"));
    }
}
```

`-Dname=value` goes **before** the class name. `COURSE=CSE101 java ...` sets a variable for one command on macOS; on **Windows**: `set COURSE=CSE101` then `java ...`. On Windows `path.separator` is `;`.

System properties (-D) and environment variables

codeoutput/Props/Props.java — Java

```java
public class Props {
    public static void main(String[] args) {
        String g = System.getProperty("greeting", "Hello"); // -D
        String c = System.getenv("COURSE");                 // env
        System.out.println(g + " from " + (c == null ? "?" : c));
        System.out.println(System.getProperty("os.name")
                + ", path.separator = "
                + System.getProperty("path.separator"));
    }
}
```

Terminal — codeoutput/Props.txt

```
$ javac Props.java
$ java Props
Hello from ?
Linux, path.separator = :
$ java -Dgreeting=Merhaba Props
Merhaba from ?
Linux, path.separator = :
$ COURSE=CSE101 java -Dgreeting=Selam Props
Selam from CSE101
Linux, path.separator = :
```

`-Dname=value` goes **before** the class name. `COURSE=CSE101 java ...` sets a variable for one command on macOS; on **Windows**: `set COURSE=CSE101` then `java ...`. On Windows `path.separator` is `;`.

Packages and Many Files

A small project with a package

codeoutput/Packages/src/edu/yeditepe/shop/Main.ja
Java

```java
package edu.yeditepe.shop;

public class Main {
    public static void main(String[] args) {
        Cart cart = new Cart();
        cart.add(new Item("pencil", 12.50));
        cart.add(new Item("notebook", 45.00));
        System.out.printf("%d items, total %.2f TL%n", cart.size
        (), cart.total());
    }
}
```

codeoutput/Packages/src/edu/yeditepe/shop/Item.ja
Java

```java
package edu.yeditepe.shop;

public record Item(String name, double price) { }
```

codeoutput/Packages/src/edu/yeditepe/shop/Cart.
Java

```java
package edu.yeditepe.shop;

import java.util.ArrayList;
import java.util.List;

public class Cart {
    private final List<Item> items = new ArrayList<>();

    public void add(Item item) { items.add(item); }
    public int size() { return items.size(); }

    public double total() {
        double t = 0;
        for (Item i : items) t += i.price();
        return t;
    }
}
```

Compiling and running a package

Terminal codeoutput/Packages.txt

```
$ find src -type f | sort
src/edu/yeditepe/shop/Cart.java
src/edu/yeditepe/shop/Item.java
src/edu/yeditepe/shop/Main.java
$ javac -d out src/edu/yeditepe/shop/*.java
$ find out -type f | sort
out/edu/yeditepe/shop/Cart.class
out/edu/yeditepe/shop/Item.class
out/edu/yeditepe/shop/Main.class
$ java -cp out edu.yeditepe.shop.Main
2 items, total 57.50 TL
$ java -cp out Main
Error: Could not find or load main class Main
Caused by: java.lang.ClassNotFoundException: Main
$ java -cp out/edu/yeditepe/shop Main
Error: Could not find or load main class Main
Caused by: java.lang.NoClassDefFoundError: Main (wrong name: edu/yeditepe/shop/Main)
```

`-d out` creates the package folders. Run with the **full name**; neither the short name nor the package folder works (“wrong name”). **Windows**: `javac -d out src\edu\yeditepe\shop\*.java`.

Many source files: `@argfile` and `-sourcepath`

Terminal codeoutput/ManyFiles.txt

```
$ find src -name "*.java" > sources.txt
$ cat sources.txt
src/edu/yeditepe/shop/Item.java
src/edu/yeditepe/shop/Cart.java
src/edu/yeditepe/shop/Main.java
$ javac -d out @sources.txt
$ rm -r out
$ javac -d out -sourcepath src src/edu/yeditepe/shop/Main.java
$ find out -type f | sort
out/edu/yeditepe/shop/Cart.class
out/edu/yeditepe/shop/Item.class
out/edu/yeditepe/shop/Main.class
```

`@sources.txt` reads the file names from a file. With `-sourcepath src`, `javac` finds and compiles `Cart` and `Item` by itself. **Windows**: `dir /s /b src\*.java > sources.txt`.

Packaging in JARs

Making a JAR, then an executable JAR

Terminal codeoutput/Jar.txt

```
$ javac -d out src/edu/yeditepe/shop/*.java
$ jar --create --file shop.jar -C out .
$ jar --list --file shop.jar
META-INF/
META-INF/MANIFEST.MF
edu/
edu/yeditepe/
edu/yeditepe/shop/
edu/yeditepe/shop/Cart.class
edu/yeditepe/shop/Item.class
edu/yeditepe/shop/Main.class
$ java -cp shop.jar edu.yeditepe.shop.Main
2 items, total 57.50 TL
$ java -jar shop.jar
no main manifest attribute, in shop.jar
$ jar --create --file shop.jar --main-class edu.yeditepe.shop.Main -C out .
$ java -jar shop.jar
2 items, total 57.50 TL
$ unzip -p shop.jar META-INF/MANIFEST.MF
Manifest-Version: 1.0
Created-By: 21.0.12.1 (Ubuntu)
Main-Class: edu.yeditepe.shop.Main
```

Without `-main-class`, `java -jar` does not know where to start. `unzip -p` prints a file from inside the JAR (**Windows**: `tar -xOf shop.jar META-INF/MANIFEST.MF`, or open the JAR with any ZIP tool).

The short (classic) jar options

Terminal codeoutput/JarShort.txt

```
$ javac -d out src/edu/yeditepe/shop/*.java
$ jar cfe shop.jar edu.yeditepe.shop.Main -C out .
$ jar tf shop.jar
META-INF/
META-INF/MANIFEST.MF
edu/
edu/yeditepe/
edu/yeditepe/shop/
edu/yeditepe/shop/Cart.class
edu/yeditepe/shop/Item.class
edu/yeditepe/shop/Main.class
$ jar tvf shop.jar
     0 Tue Oct 13 16:52:34 TRT 2026 META-INF/
    93 Tue Oct 13 16:52:34 TRT 2026 META-INF/MANIFEST.MF
     0 Tue Oct 13 16:52:34 TRT 2026 edu/
     0 Tue Oct 13 16:52:34 TRT 2026 edu/yeditepe/
     0 Tue Oct 13 16:52:34 TRT 2026 edu/yeditepe/shop/
   887 Tue Oct 13 16:52:34 TRT 2026 edu/yeditepe/shop/Cart.class
  1266 Tue Oct 13 16:52:34 TRT 2026 edu/yeditepe/shop/Item.class
   917 Tue Oct 13 16:52:34 TRT 2026 edu/yeditepe/shop/Main.class
$ java -jar shop.jar
2 items, total 57.50 TL
```

`c` create, `t` list, `x` extract, `v` verbose, `f` file name follows, `e` entry point (main class), like the old Unix `tar`. `jar cfe` = `jar -create -file ... -main-class ...`

Resources inside a JAR

codeoutput/Resources/src/Banner.java **Java**

```java
import java.io.IOException;
import java.io.InputStream;
import java.nio.file.Files;
import java.nio.file.Path;

public class Banner {
    public static void main(String[] args) throws IOException {
        // 1. a resource: searched on the class path
        try (InputStream in = Banner.class.getResourceAsStream("/banner.txt")) {
            System.out.print("resource: " + new String(in.readAllBytes()));
        }
        // 2. a file: relative to the CURRENT folder
        Path p = Path.of("banner.txt");
        String f = Files.exists(p) ? Files.readString(p).strip() : "not found";
        System.out.println("file    : " + f);
    }
}
```

`getResourceAsStream("/banner.txt")` searches the **class path**, so it works from any folder. A plain file path is relative to the **current folder**: run elsewhere, the file is gone.

Resources inside a JAR

codeoutput/Resources/src/Banner.java — Java

```java
import java.io.IOException;
import java.io.InputStream;
import java.nio.file.Files;
import java.nio.file.Path;

public class Banner {
    public static void main(String[] args) throws IOException {
        // 1. a resource: searched on the class path
        try (InputStream in = Banner.class.getResourceAsStream("/banner.txt")) {
            System.out.print("resource: " + new String(in.readAllBytes()));
        }
        // 2. a file: relative to the CURRENT folder
        Path p = Path.of("banner.txt");
        String f = Files.exists(p) ? Files.readString(p).strip() : "not found";
        System.out.println("file    : " + f);
    }
}
```

Terminal — codeoutput/Resources.txt

```
$ javac -d out src/Banner.java
$ jar --create --file banner.jar --main-class
      Banner -C out . -C res .
$ jar --list --file banner.jar
META-INF/
META-INF/MANIFEST.MF
Banner.class
banner.txt
$ java -jar banner.jar
resource: Yeditepe University
file    : Yeditepe University (a file)
$ mkdir elsewhere
$ cd elsewhere
$ java -jar ../banner.jar
resource: Yeditepe University
file    : not found
```

`getResourceAsStream("/banner.txt")` searches the **class path**, so it works from any folder. A plain file path is relative to the **current folder**: run elsewhere, the file is gone.

Using a library JAR

codeoutput/Library/lib/edu/yeditepe/greet/Greeter.java — Java

```java
package edu.yeditepe.greet;

/** Greets people in Turkish or English. */
public class Greeter {
    /** @return a greeting for {@code name} */
    public static String greet(String name, boolean turkish) {
        return (turkish ? "Merhaba, " : "Hello, ") + name + "!";
    }
}
```

codeoutput/Library/App.java — Java

```java
import edu.yeditepe.greet.Greeter;

public class App {
    public static void main(String[] args) {
        System.out.println(Greeter.greet("Ayşe", true));
        System.out.println(Greeter.greet("Bob", false));
    }
}
```

The library must be on the class path **twice**: when compiling (`javac -cp`) and when running (`java -cp`). **Windows**: `java -cp out;greet.jar App`.

Using a library JAR

codeoutput/Library/lib/edu/yeditepe/greet/Greeter.java

```java
package edu.yeditepe.greet;

/** Greets people in Turkish or English. */
public class Greeter {
    /** @return a greeting for {@code name} */
    public static String greet(String name, boolean turkish) {
        return (turkish ? "Merhaba, " : "Hello, ") + name + "!";
    }
}
```

codeoutput/Library/App.java

```java
import edu.yeditepe.greet.Greeter;

public class App {
    public static void main(String[] args) {
        System.out.println(Greeter.greet("Ayşe", true));
        System.out.println(Greeter.greet("Bob", false));
    }
}
```

Terminal — codeoutput/Library.txt

```
$ javac -d libout lib/edu/yeditepe/greet/Greeter.java
$ jar --create --file greet.jar -C libout .
$ javac -d out -cp greet.jar App.java
$ java -cp out:greet.jar App
Merhaba, Ayşe!
Hello, Bob!
$ java -cp out App 2>&1 | head -3
Exception in thread "main" java.lang.NoClassDefFoundError: edu/
        yeditepe/greet/Greeter
         at App.main(App.java:5)
Caused by: java.lang.ClassNotFoundException: edu.yeditepe.greet.
        Greeter
```

The library must be on the class path **twice**: when compiling (`javac -cp`) and when running (`java -cp`). **Windows**: `java -cp out;greet.jar App`.

Errors and Other Tools

Compile-time errors

codeoutput/Errors/Broken.java — Java

```java
public class Broken {
    public static void main(String[] args) {
        int count = 10
        String s = count;
        System.out.println(s);
    }
}
```

Terminal — codeoutput/Errors.txt

```
$ javac Broken.java
Broken.java:3: error: ';' expected
        int count = 10
                    ^
1 error
```

`javac` reports **file:line**, a message and a ^ under the spot. Here it stops after the syntax error; fix it, compile again, and the type error on line 4 appears.

Run-time errors: reading a stack trace

codeoutput/Crash/Crash.java **Java**

```java
public class Crash {
    static int average(int[] a) {
        int sum = 0;
        for (int x : a) sum += x;
        return sum / a.length;
    }

    public static void main(String[] args) {
        System.out.println(average(new int[] {4, 8}));
        System.out.println(average(new int[] {}));
    }
}
```

Read the trace **top-down**: the exception, then where it happened (`Crash.java:5` in `average`), then who called it (`Crash.java:10` in `main`). The exit code is 1.

Run-time errors: reading a stack trace

codeoutput/Crash/Crash.java — Java

```java
public class Crash {
    static int average(int[] a) {
        int sum = 0;
        for (int x : a) sum += x;
        return sum / a.length;
    }

    public static void main(String[] args) {
        System.out.println(average(new int[] {4, 8}));
        System.out.println(average(new int[] {}));
    }
}
```

Terminal — codeoutput/Crash.txt

```
$ javac Crash.java
$ java Crash
6
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at Crash.average(Crash.java:5)
        at Crash.main(Crash.java:10)
```

Read the trace **top-down**: the exception, then where it happened (`Crash.java:5` in `average`), then who called it (`Crash.java:10` in `main`). The exit code is 1.

javap: looking inside a .class file

Terminal codeoutput/Javap.txt

```
$ javac Hello.java
$ javap -c Hello
Compiled from "Hello.java"
public class Hello {
  public Hello();
    Code:
       0: aload_0
       1: invokespecial #1                  // Method java/lang/Object."<init>":()V
       4: return

  public static void main(java.lang.String[]);
    Code:
       0: getstatic     #7                  // Field java/lang/System.out:Ljava/io/PrintStream;
       3: ldc           #13                 // String Hello, World!
       5: invokevirtual #15                 // Method java/io/PrintStream.println:(Ljava/lang/String;)V
       8: return
}
$ javap -v Hello | grep major
  major version: 65
```

The bytecode: `getstatic` fetches `System.out`, `ldc` loads the string, `invokevirtual` calls `println`. **Major version 65** means the file needs Java 21 or newer.

jshell: Java without a class

Type `jshell`, then expressions one by one (`/exit` to leave); or give it a script file:

Terminal codeoutput/JShell.txt

```
$ cat try.jsh
int x = 6 * 7;
System.out.println("x = " + x);
String s = "Yeditepe";
System.out.println(s.toUpperCase() + " has " + s.length() + " letters");
System.out.println(Math.sqrt(2));
/exit
$ jshell -q try.jsh
x = 42
YEDITEPE has 8 letters
1.4142135623730951
```

Good for trying an API before writing a program. `-q` (quiet) skips the welcome text.

javadoc: documentation from comments

codeoutput/Docs/lib/edu/yeditepe/greet/Greeter.ja
Java

```java
package edu.yeditepe.greet;

/** Greets people in Turkish or English. */
public class Greeter {
    private Greeter() { }          // no objects needed

    /**
     * Builds a greeting.
     * @param name    who to greet
     * @param turkish {@code true} for Turkish
     * @return the greeting
     */
    public static String greet(String name, boolean turkish) {
        return (turkish ? "Merhaba, " : "Hello, ") + name + "!";
    }
}
```

Open `docs/index.html` in a browser: the same format as the official Java API documentation.

javadoc: documentation from comments

codeoutput/Docs/lib/edu/yeditepe/greet/Greeter.ja

Java

```java
package edu.yeditepe.greet;

/** Greets people in Turkish or English. */
public class Greeter {
    private Greeter() { }          // no objects needed

    /**
     * Builds a greeting.
     * @param name    who to greet
     * @param turkish {@code true} for Turkish
     * @return the greeting
     */
    public static String greet(String name, boolean turkish) {
        return (turkish ? "Merhaba, " : "Hello, ") + name + "!";
    }
}
```

Terminal codeoutput/Docs.txt

```
$ javadoc -quiet -d docs lib/edu/yeditepe/greet/Greeter.java
$ ls docs | xargs -n 4
allclasses-index.html allpackages-index.html copy.svg edu
element-list help-doc.html index-all.html index.html
legal link.svg member-search-index.js module-search-index.js
overview-tree.html package-search-index.js resources script-dir
script.js search-page.js search.html search.js
stylesheet.css tag-search-index.js type-search-index.js
$ ls docs/edu/yeditepe/greet
Greeter.html
package-summary.html
package-tree.html
```

Open `docs/index.html` in a browser: the same format as the official Java API documentation.

jdeps: what does a JAR depend on?

Terminal codeoutput/Jdeps.txt

```
$ javac -d out src/edu/yeditepe/shop/*.java
$ jar --create --file shop.jar --main-class edu.yeditepe.shop.Main -C out .
$ jdeps -summary shop.jar
shop.jar -> java.base
$ jdeps --list-deps shop.jar
   java.base
$ jdeps --print-module-deps shop.jar
java.base
$ javac -d rout Report.java
$ jar --create --file report.jar -C rout .
$ jdeps --print-module-deps report.jar
java.base,java.sql
```

The shop only needs `java.base`. `Report` uses `java.sql.Date` and `Logger`, yet only `java.sql` is listed: `java.sql` already requires `java.logging`. This is the list `jlink` wants (junior slides).

Summary

Cheat sheet

Command	Does
`javac Hello.java`	compile into the current folder
`javac -d out -sourcepath src src/a/b/Main.java`	compile a project into `out`
`java Hello` / `java -cp out a.b.Main`	run a class
`java Hello.java`	compile in memory and run (Java 11+)
`java -Dkey=value -cp out a.b.Main arg1 arg2`	property, then class, then arguments
`jar -create -file app.jar -main-class a.b.Main -C out .`	executable JAR (`jar cfe`)
`jar -list -file app.jar` (`jar tf`)	list contents
`java -jar app.jar`	run an executable JAR
`java -cp out:lib/x.jar Main` (**Windows**: `;`)	use a library
`javap -c Hello`, `jshell`, `javadoc -d docs ...`, `jdeps app.jar`	tools

Exercises

1. Write `Upper.java` that prints its arguments in upper case; run it with and without quotes.
2. Make `Sum` read from a file and write the result to another file using only redirection.
3. Give `Check` a third exit code (3 for zero) and test it in a one-line `&&`/`||` command.
4. Add a class `Discount` to the shop project; compile everything with one `-sourcepath` command.
5. Package the shop as an executable JAR; run it from a different folder.
6. Put a `prices.txt` resource into the JAR and read it with `getResourceAsStream`.
7. Split `Greeter` into a library JAR with `javadoc`; use it from two different programs.

Questions?